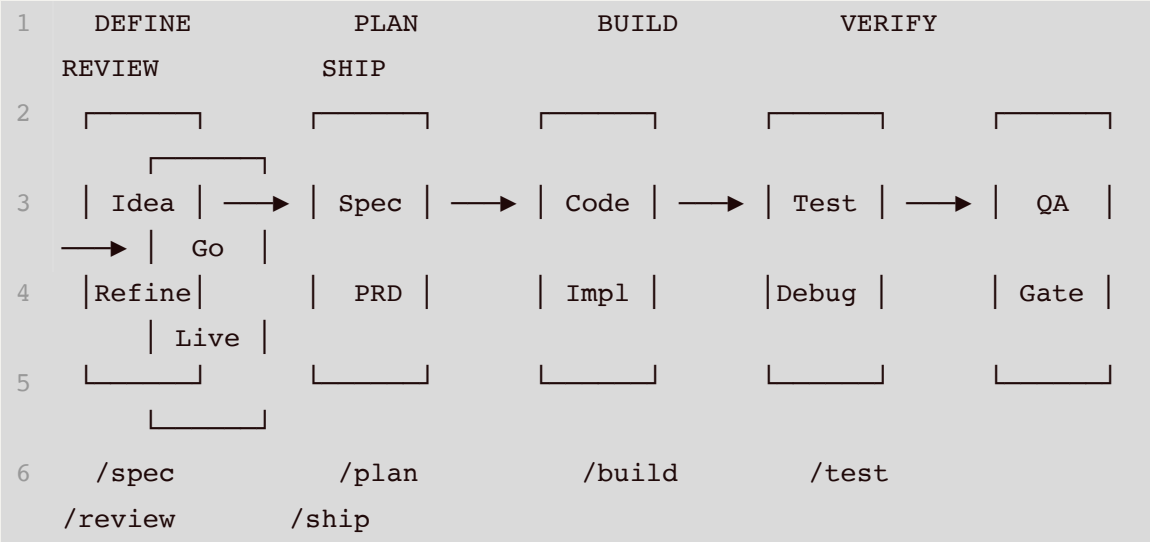
Production-grade engineering skills for AI coding agents.md

> 📎 来源: *unknown*

Agent Skills

Production-grade engineering skills for AI coding agents.

Skills encode the workflows, quality gates, and best practices that senior engineers use when building software. These ones are packaged so AI agents follow them consistently across every phase of development.



Commands

7 slash commands that map to the development lifecycle. Each one activates the right skills automatically.

WHAT YOU'RE DOING	COMMAND	KEY PRINCIPLE
Define what to build	`/spec`	Spec before code
Plan how to build it	`/plan`	Small, atomic tasks
Build incrementally	`/build`	One slice at a time
Prove it works	`/test`	Tests are proof
Review before merge	`/review`	Improve code health
Simplify the code	`/code-simplify`	Clarity over cleverness
Ship to production	`/ship`	Faster is safer

Skills also activate automatically based on what you're doing — designing an API triggers `api-and-interface-design`, building UI triggers `frontend-ui-engineering`, and so on.

Quick Start

▶ Claude Code (recommended)

▶ Cursor

▶ Windsurf

▶ GitHub Copilot

▶ Codex / Other Agents

All 19 Skills

The commands above are the entry points. Under the hood, they activate these 19 skills — each one a structured workflow with steps, verification gates, and anti-rationalization tables. You can also reference any skill directly.

Define - Clarify what to build

SKILL	WHAT IT DOES	USE WHEN
idea-refine	Structured divergent/convergent thinking to turn vague ideas into concrete proposals	You have a rough concept that needs exploration
spec-driven-development	Write a PRD covering objectives, commands, structure, code style, testing, and boundaries before any code	Starting a new project, feature, or significant change

Plan - Break it down

SKILL	WHAT IT DOES	USE WHEN
planning-and-task-breakdown	Decompose specs into small, verifiable tasks with acceptance criteria and dependency ordering	You have a spec and need implementable units

Build - Write the code

SKILL	WHAT IT DOES	USE WHEN
incremental-implementation	Thin vertical slices - implement, test, verify, commit. Feature flags, safe defaults, rollback-friendly changes	Any change touching more than one file
context-engineering	Feed agents the right information at the right time - rules files, context packing, MCP integrations	Starting a session, switching tasks, or when output quality drops
frontend-ui-engineering	Component architecture, design systems, state management, responsive design, WCAG 2.1 AA accessibility	Building or modifying user-facing interfaces
api-and-interface-design	Contract-first design, Hyrum's Law, One-Version Rule, error semantics, boundary validation	Designing APIs, module boundaries, or public interfaces

Verify - Prove it works

SKILL	WHAT IT DOES	USE WHEN
test-driven-development	Red-Green-Refactor, test pyramid (80/15/5), test sizes, DAMP over DRY, Beyonce Rule, browser testing	Implementing logic, fixing bugs, or changing behavior
browser-testing-with-devtools	Chrome DevTools MCP for live runtime data - DOM inspection, console logs, network traces, performance profiling	Building or debugging anything that runs in a browser
debugging-and-error-recovery	Five-step triage: reproduce, localize, reduce, fix, guard. Stop-the-line rule, safe fallbacks	Tests fail, builds break, or behavior is unexpected

Review - Quality gates before merge

SKILL	WHAT IT DOES	USE WHEN
code-review-and-quality	Five-axis review, change sizing (~100 lines), severity labels (Nit/Optional/FYI), review speed norms, splitting strategies	Before merging any change
code-simplification	Chesterton's Fence, Rule of 500, reduce complexity while preserving exact behavior	Code works but is harder to read or maintain than it should be
security-and-hardening	OWASP Top 10 prevention, auth patterns, secrets management, dependency auditing, three-tier boundary system	Handling user input, auth, data storage, or external integrations
performance-optimization	Measure-first approach - Core Web Vitals targets, profiling workflows, bundle analysis, anti-pattern detection	Performance requirements exist or you suspect regressions

Ship - Deploy with confidence

SKILL	WHAT IT DOES	USE WHEN
git-workflow-and-versioning	Trunk-based development, atomic commits, change sizing (~100 lines), the commit-as-save-point pattern	Making any code change (always)
ci-cd-and-automation	Shift Left, Faster is Safer, feature flags, quality gate pipelines, failure feedback loops	Setting up or modifying build and deploy pipelines
deprecation-and-migration	Code-as-liability mindset, compulsory vs advisory deprecation, migration patterns, zombie code removal	Removing old systems, migrating users, or sunsetting features
documentation-and-adrs	Architecture Decision Records, API docs, inline documentation standards - document the *why*	Making architectural decisions, changing APIs, or shipping features
shipping-and-launch	Pre-launch checklists, feature flag lifecycle, staged rollouts, rollback procedures, monitoring setup	Preparing to deploy to production

---

Agent Personas

Pre-configured specialist personas for targeted reviews:

AGENT	ROLE	PERSPECTIVE
code-reviewer	Senior Staff Engineer	Five-axis code review with "would a staff engineer approve this?" standard
test-engineer	QA Specialist	Test strategy, coverage analysis, and the Prove-It pattern
security-auditor	Security Engineer	Vulnerability detection, threat modeling, OWASP assessment

---

Reference Checklists

Quick-reference material that skills pull in when needed:

REFERENCE	COVERS
testing-patterns.md	Test structure, naming, mocking, React/API/E2E examples, anti-patterns
security-checklist.md	Pre-commit checks, auth, input validation, headers, CORS, OWASP Top 10
performance-checklist.md	Core Web Vitals targets, frontend/backend checklists, measurement commands
accessibility-checklist.md	Keyboard nav, screen readers, visual design, ARIA, testing tools

---

How Skills Work

Every skill follows a consistent anatomy:

```
┌──────────────────────────────────────────────┐
│  SKILL.md                                    │
│                                              │
│  ┌─ Frontmatter ──────────────────────────┐  │
│  │ name: lowercase-hyphen-name            │  │
│  │ description: Use when [trigger]        │  │
│  └────────────────────────────────────────┘  │
│                                              │
│  Overview          → What this skill does    │
│  When to Use       → Triggering conditions   │
│  Process           → Step-by-step workflow   │
│  Rationalizations  → Excuses + rebuttals     │
│  Red Flags         → Signs something's wrong │
│  Verification      → Evidence requirements   │
└──────────────────────────────────────────────┘
```

Key design choices:

- **Process, not prose.** Skills are workflows agents follow, not reference docs they read. Each has steps, checkpoints, and exit criteria.
- **Anti-rationalization.** Every skill includes a table of common excuses agents use to skip steps (e.g., "I'll add tests later") with documented counter-arguments.
- **Verification is non-negotiable.** Every skill ends with evidence requirements - tests passing, build output, runtime data. "Seems right" is never sufficient.

- **Progressive disclosure.** The `SKILL.md` is the entry point. Supporting references load only when needed, keeping token usage minimal.

Project Structure

```
agent-skills/
├── skills/                            # 19 core skills (SKILL.md per directory)
│   ├── idea-refine/                   #   Define
│   ├── spec-driven-development/       #   Define
│   ├── planning-and-task-breakdown/   #   Plan
│   ├── incremental-implementation/    #   Build
│   ├── context-engineering/           #   Build
│   ├── frontend-ui-engineering/       #   Build
│   ├── api-and-interface-design/      #   Build
│   ├── test-driven-development/       #   Verify
│   ├── browser-testing-with-devtools/ #   Verify
│   ├── debugging-and-error-recovery/  #   Verify
│   ├── code-review-and-quality/       #   Review
│   ├── code-simplification/          #   Review
│   ├── security-and-hardening/        #   Review
│   ├── performance-optimization/      #   Review
│   ├── git-workflow-and-versioning/   #   Ship
│   ├── ci-cd-and-automation/          #   Ship
│   ├── deprecation-and-migration/     #   Ship
│   ├── documentation-and-adrs/        #   Ship
│   ├── shipping-and-launch/           #   Ship
│   └── using-agent-skills/            #   Meta: how to use this pack
├── agents/                            # 3 specialist personas
├── references/                        # 4 supplementary checklists
├── hooks/                             # Session lifecycle hooks
├── .claude/commands/                  # 7 slash commands
└── docs/                              # Setup guides per tool
```

Why Agent Skills?

AI coding agents default to the shortest path - which often means skipping specs, tests, security reviews, and the practices that make software reliable. Agent Skills gives agents structured workflows that enforce the same discipline senior engineers bring to production code.

Each skill encodes hard-won engineering judgment: *when* to write a spec, *what* to test, *how* to review, and *when* to ship. These aren't generic prompts - they're the kind of opinionated, process-driven workflows that separate production-quality work from prototype-quality work.

Skills bake in best practices from Google's engineering culture — including concepts from Software Engineering at Google and Google's engineering practices guide. You'll find Hyrum's Law in API design, the Beyonce Rule and test pyramid in testing, change sizing and review speed norms in code review, Chesterton's Fence in simplification, trunk-based development in git workflow, Shift Left and feature flags in CI/CD, and a dedicated deprecation skill treating code as a liability. These aren't abstract principles — they're embedded directly into the step-by-step workflows agents follow.

Contributing

Skills should be **specific** (actionable steps, not vague advice), **verifiable** (clear exit criteria with evidence requirements), **battle-tested** (based on real workflows), and **minimal** (only what's needed to guide the agent).

See docs/skill-anatomy.md for the format specification and CONTRIBUTING.md for guidelines.

License

MIT - use these skills in your projects, teams, and tools.